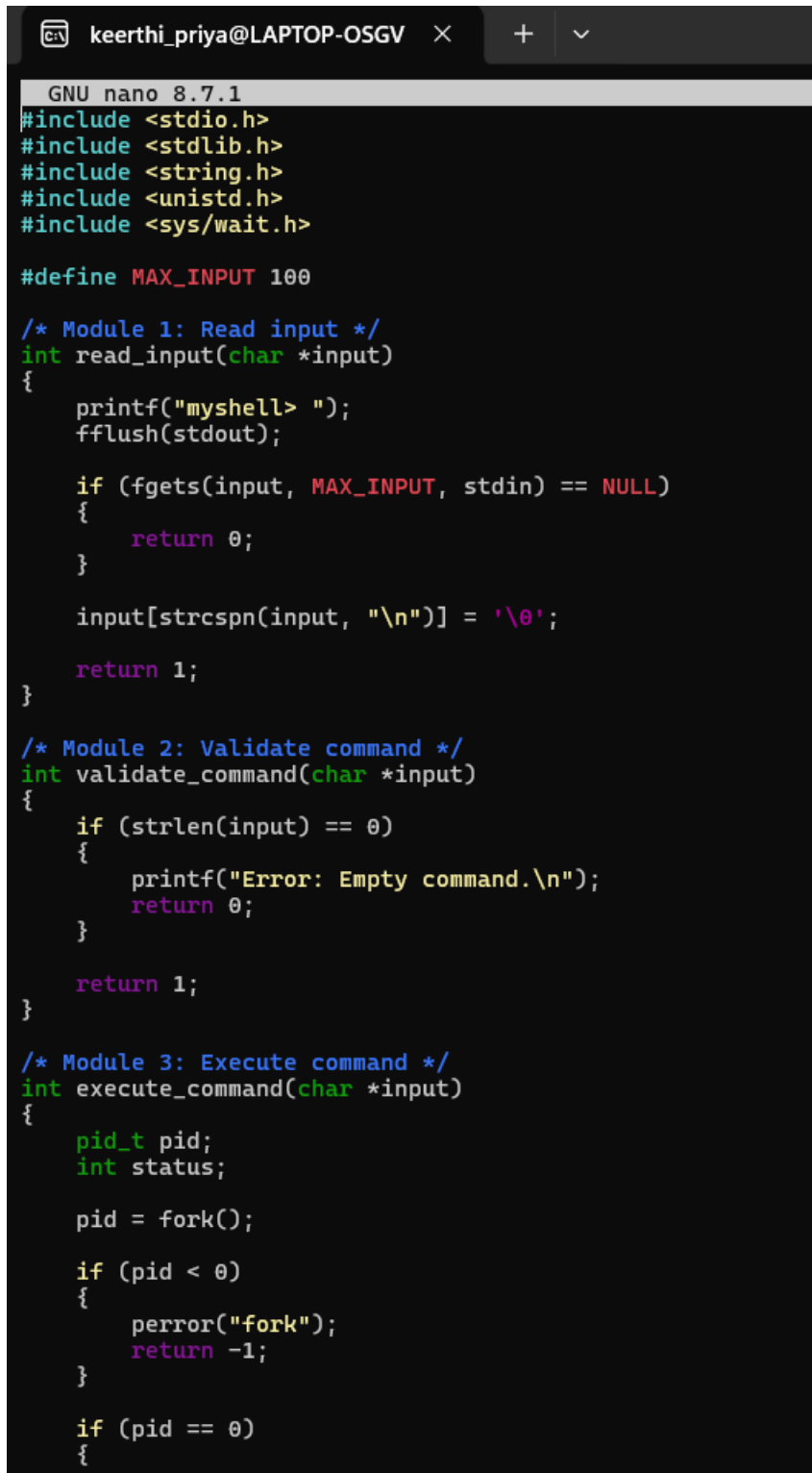


SKILL WEEK – 11

Session – 21

TASK 1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor 4 Components

A screenshot of a terminal window with a dark background. The window title bar shows 'keerthi_priya@LAPTOP-OSGV' and standard window controls. The terminal displays the GNU nano 8.7.1 editor with C code. The code includes headers for stdio, stdlib, string, unistd, and sys/wait. It defines MAX_INPUT as 100 and contains three modules: read_input, validate_command, and execute_command. The read_input module prints 'myshell>' and reads input. The validate_command module checks for empty input. The execute_command module forks a child process to execute the command.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 100

/* Module 1: Read input */
int read_input(char *input)
{
    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, MAX_INPUT, stdin) == NULL)
    {
        return 0;
    }

    input[strcspn(input, "\n")] = '\0';

    return 1;
}

/* Module 2: Validate command */
int validate_command(char *input)
{
    if (strlen(input) == 0)
    {
        printf("Error: Empty command.\n");
        return 0;
    }

    return 1;
}

/* Module 3: Execute command */
int execute_command(char *input)
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return -1;
    }

    if (pid == 0)
    {

```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

if (pid == 0)
{
    char *args[10];
    int count = 0;

    char *token = strtok(input, " ");

    while (token != NULL && count < 9)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    if (count == 0)
    {
        exit(1);
    }

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

if (waitpid(pid, &status, 0) == -1)
{
    perror("waitpid");
    return -1;
}

if (WIFEXITED(status))
{
    printf("Command exited with status %d\n",
           WEXITSTATUS(status));
}
else
{
    printf("Command did not exit normally.\n");
}

return 0;
}

/* Module 4: Main control */
int main()
{
    char input[MAX_INPUT];

    printf("Session 21 - Modular Shell Integration\n");
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

    }

    if (WIFEXITED(status))
    {
        printf("Command exited with status %d\n",
               WEXITSTATUS(status));
    }
    else
    {
        printf("Command did not exit normally.\n");
    }

    return 0;
}

/* Module 4: Main control */
int main()
{
    char input[MAX_INPUT];

    printf("Session 21 - Modular Shell Integration\n");
    printf("-----\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        if (!read_input(input))
        {
            printf("\nInput error or EOF.\n");
            break;
        }

        if (strcmp(input, "exit") == 0)
        {
            printf("Shell exiting successfully.\n");
            break;
        }

        if (!validate_command(input))
        {
            continue;
        }

        if (execute_command(input) != 0)
        {
            printf("Error: Command execution failed.\n");
        }
    }

    return 0;
}
```

```

keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cd /mnt/c/JavaPrograms
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session21_task1.c

keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session21_task1.c -o session21_task1
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session21_task1
Session 21 - Modular Shell Integration
-----
Type 'exit' to quit.

myshell> ls
Arithmetic.class      ObjectDemo.java      session11_task1
Arithmetic.java       Operator.class        session11_task1.c
Arithmetic.txt        Operator.java         session11_task2
Array_output.java     Peven.class          session11_task2.c
Calci.class           Peven.java           session12_task1
Calci.java            Positive.class        session12_task1.c
Datatypes.class       Positive.java         session12_task2
Datatypes.java        Positive.class        session12_task2.c
Demo.class            Print.class          session19_task1
Demo.java            ScanDemo.class       session19_task1.c
Divmod.class          ScanDemo.java        session19_task2
Divmod.java          StringDemo.class     session19_task2.c
Edowhile.class       StringDemo.java      session20_task1
Edowhile.java        SubMul.class         session20_task1.c
Efor.class           SubMul.java          session20_task2
Efor.java            SumDemo.class        session20_task2.c
Efor.txt             SumDemo.java         session21_task1
Equal.class          SumDemo3.java        session21_task1.c
Equal.java           SumMatrix.class      session3_task2
Even.class           SumMatrix.java       session3_task2.c
Ewhile.class         Sumeven.class        session4_task1
Ewhile.java          Sumeven.java         session4_task1.c
Ewhile.java          Sumodd.class         session4_task2
FactorialDemo.class  Sumodd.java          session4_task2.c
FactorialDemo.java   TestDemo.class       session5_task1
Floating.class       TestDemo.java        session5_task1.c
Grades.class         Three.class          session5_task2
Grades.java         Three.java           session5_task2.c
'Int variable.java'  child.class          session6_task1
'Int variables.java' even.java            session6_task1.c
Jhansi.class         floating.java        session6_task2
Jhansi.java          hardlink.txt         session6_task2.c
Keerthi.Html         input.txt            session7_task1
Leapyear.class       mmap_copy.txt        session7_task1.c
Leapyear.java        original.txt          session7_task2
MatrixTranspose.class p-6.c               session7_task2.c
MatrixTranspose.java p1                  symlink.txt
MethodOverLoading.java p62                vinnu.class
MethodOverLoading.txt parent.class         vinnu.java
Min.java             print.java          vote.class

```

```

Ewhile.class         Sumeven.class        session4_task1
Ewhile.java          Sumeven.java         session4_task1.c
Ewhile.java          Sumodd.class         session4_task2
FactorialDemo.class  Sumodd.java          session4_task2.c
FactorialDemo.java   TestDemo.class       session5_task1
Floating.class       TestDemo.java        session5_task1.c
Grades.class         Three.class          session5_task2
Grades.java         Three.java           session5_task2.c
'Int variable.java'  child.class          session6_task1
'Int variables.java' even.java            session6_task1.c
Jhansi.class         floating.java        session6_task2
Jhansi.java          hardlink.txt         session6_task2.c
Keerthi.Html         input.txt            session7_task1
Leapyear.class       mmap_copy.txt        session7_task1.c
Leapyear.java        original.txt          session7_task2
MatrixTranspose.class p-6.c               session7_task2.c
MatrixTranspose.java p1                  symlink.txt
MethodOverLoading.java p62                vinnu.class
MethodOverLoading.txt parent.class         vinnu.java
Min.java             print.java          vote.class
New.class            read_write_copy.txt  vote.java
New.java             session10_task2
ObjectDemo.class     session10_task2.c
Command exited with status 0
myshell> pwd
/mnt/c/JavaPrograms
Command exited with status 0
myshell> Hello
execvp: No such file or directory
Command exited with status 1
myshell> Empty command
execvp: No such file or directory
Command exited with status 1
myshell> exit
Shell exiting successfully.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |

```

TASK 2: TO Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

```
keerthi_priya@LAPTOP-OSGV X + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_INPUT 100

/* Log errors into a file */
void log_error(const char *message)
{
    FILE *file = fopen("shell_errors.log", "a");
    if (file == NULL)
    {
        perror("fopen");
        return;
    }

    fprintf(file, "ERROR: %s\n", message);
    fclose(file);
}

/* Validate input syntax */
int validate_syntax(char *input)
{
    if (strlen(input) == 0)
    {
        printf("Error: Empty command.\n");
        log_error("Empty command");
        return 0;
    }

    if (strstr(input, "&&") != NULL ||
        strstr(input, "||") != NULL)
    {
        printf("Error: Invalid syntax.\n");
        log_error("Invalid command syntax");
        return 0;
    }

    return 1;
}

/* Execute command */
int execute_command(char *input)
{
    pid_t pid;
    int status;

    pid = fork();
    if (pid < 0)
    {
        perror("fork");
        log_error("fork failed");
        return -1;
    }
}
```

```
keerthi_priya@LAPTOP-OSGV X + v
GNU nano 8.7.1
        log_error("fork failed");
        return -1;
    }

    if (pid == 0)
    {
        char *args[10];
        int count = 0;

        char *token = strtok(input, " ");
        while (token != NULL && count < 9)
        {
            args[count++] = token;
            token = strtok(NULL, " ");
        }

        args[count] = NULL;

        if (count == 0)
        {
            exit(1);
        }

        execvp(args[0], args);

        fprintf(stderr, "Runtime Error: Cannot execute '%s'\n",
            args[0]);

        log_error("Command execution failed");
        exit(127);
    }

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid");
        log_error("waitpid failed");
        return -1;
    }

    if (WIFEXITED(status))
    {
        int exit_status = WEXITSTATUS(status);

        printf("Command exited with status %d\n",
            exit_status);

        if (exit_status != 0)
        {
            log_error("Command returned a non-zero status");
        }
    }

    return 0;
}

int main()
{
    char input[MAX_INPUT];
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
    }
    return -1;
}

if (WIFEXITED(status))
{
    int exit_status = WEXITSTATUS(status);

    printf("Command exited with status %d\n",
        exit_status);

    if (exit_status != 0)
    {
        log_error("Command returned a non-zero status");
    }
}

return 0;
}

int main()
{
    char input[MAX_INPUT];

    printf("Session 21 - Runtime Error Handling\n");
    printf("-----\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, MAX_INPUT, stdin) == NULL)
        {
            printf("\nInput error or EOF.\n");
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Shell exiting safely.\n");
            break;
        }

        if (!validate_syntax(input))
        {
            printf("Recovering... Shell continues.\n");
            continue;
        }

        execute_command(input);

        printf("Ready for next command.\n");
    }

    return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cd /mnt/c/JavaPrograms
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session21_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session21_task2.c -o session21_task2
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session21_task2
Session 21 - Runtime Error Handling
-----
Type 'exit' to quit.

myshell> pwd
/mnt/c/JavaPrograms
Command exited with status 0
Ready for next command.
myshell> abcxzy
Runtime Error: Cannot execute 'abcxyz'
Command exited with status 127
Ready for next command.
myshell> ls &&
Error: Invalid syntax.
Recovering... Shell continues.
myshell>
Error: Empty command.
Recovering... Shell continues.
myshell> exit
Shell exiting safely.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```

Session – 22

TASK 1: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

```
keerthi_priya@LAPTOP-OSGV  ×  +  ▾
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    printf("Session 22 - Valgrind Memory Analysis\n");
    printf("-----\n");

    int *numbers;
    char *message;

    /* Allocate memory */
    numbers = malloc(5 * sizeof(int));
    message = malloc(50 * sizeof(char));

    if (numbers == NULL || message == NULL)
    {
        printf("Memory allocation failed.\n");

        free(numbers);
        free(message);

        return 1;
    }

    /* Store data */
    for (int i = 0; i < 5; i++)
    {
        numbers[i] = (i + 1) * 10;
    }

    strcpy(message, "Memory allocated successfully.");

    printf("Numbers: ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", numbers[i]);
    }

    printf("\nMessage: %s\n", message);

    /* Release allocated memory */
    free(numbers);
    free(message);

    numbers = NULL;
    message = NULL;

    printf("Memory released successfully.\n");
    printf("Cleanup completed.\n");

    return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cd /mnt/c/JavaPrograms
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session22_task1.c

keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc -g session22_task1.c -o session22_task1
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session22_task1
Session 22 - Valgrind Memory Analysis
-----
Numbers: 10 20 30 40 50
Message: Memory allocated successfully.
Memory released successfully.
Cleanup completed.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```

```

Processing triggers for libc-bin (2.40-2ubuntu2.4) ...
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ valgrind --version
valgrind-3.26.0
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ valgrind --leak-check=full ./session22_task1
==2326== Memcheck, a memory error detector
==2326== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2326== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2326== Command: ./session22_task1
==2326==
Session 22 - Valgrind Memory Analysis
-----
Numbers: 10 20 30 40 50
Message: Memory allocated successfully.
Memory released successfully.
Cleanup completed.
==2326==
==2326== HEAP SUMMARY:
==2326==     in use at exit: 0 bytes in 0 blocks
==2326==   total heap usage: 3 allocs, 3 frees, 1,094 bytes allocated
==2326==
==2326== All heap blocks were freed -- no leaks are possible
==2326==
==2326== For lists of detected and suppressed errors, rerun with: -s
==2326== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |

```

TASK 2: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

```

keerthi_priya@LAPTOP-OSGV  x  +  v
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: ./session22_task2 <input>\n");
        return 1;
    }

    if (strcmp(argv[1], "hello") == 0)
    {
        printf("PASS: Hello command recognized\n");
        return 0;
    }
    else if (strcmp(argv[1], "test") == 0)
    {
        printf("PASS: Test command recognized\n");
        return 0;
    }
    else if (strcmp(argv[1], "error") == 0)
    {
        printf("ERROR: Invalid test command\n");
        return 1;
    }
    else
    {
        printf("ERROR: Unknown input\n");
        return 1;
    }
}

```

```
keerthi_priya@LAPTOP-OSGV × + ▾
GNU nano 8.7.1
#!/bin/bash

echo "Session 22 - Automated Test Report"
echo "===== "

PASS=0
FAIL=0

run_test()
{
    NAME="$1"
    INPUT="$2"
    EXPECTED="$3"

    echo ""
    echo "Test: $NAME"

    OUTPUT=$(./session22_task2 "$INPUT")

    echo "Expected: $EXPECTED"
    echo "Actual  : $OUTPUT"

    if echo "$OUTPUT" | grep -q "$EXPECTED"
    then
        echo "Result  : PASS"
        PASS=$((PASS + 1))
    else
        echo "Result  : FAIL"
        FAIL=$((FAIL + 1))
    fi
}

run_test "Hello command" "hello" "PASS"
run_test "Test command" "test" "PASS"
run_test "Invalid command" "error" "ERROR"
run_test "Unknown input" "abc" "ERROR"

echo ""
echo "===== "
echo "Test Summary"
echo "===== "
echo "Passed : $PASS"
echo "Failed : $FAIL"
echo "Total  : $((PASS + FAIL))"

if [ "$FAIL" -eq 0 ]
then
    echo "Overall Result: ALL TESTS PASSED"
else
    echo "Overall Result: SOME TESTS FAILED"
fi
```

```
keerthi_priya@LAPTOP-OSGV × + ▾
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cd /mnt/c/JavaPrograms
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session22_task2.c

keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session22_task2.c -o session22_task2
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session22_task2 hello
PASS: Hello command recognized
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano test_session22.sh

keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ chmod +x test_session22.sh
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./test_session22.sh
Session 22 - Automated Test Report
=====

Test: Hello command
Expected: PASS
Actual  : PASS: Hello command recognized
Result  : PASS

Test: Test command
Expected: PASS
Actual  : PASS: Test command recognized
Result  : PASS

Test: Invalid command
Expected: ERROR
Actual  : ERROR: Invalid test command
Result  : PASS

Test: Unknown input
Expected: ERROR
Actual  : ERROR: Unknown input
Result  : PASS

=====
Test Summary
=====
Passed : 4
Failed : 0
Total   : 4
Overall Result: ALL TESTS PASSED
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./test_session22.sh > session22_test_report.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cat session22_test_report.txt
Session 22 - Automated Test Report
=====
```

```
keerthi_priya@LAPTOP-OSGV × + ▾
Expected: ERROR
Actual : ERROR: Unknown input
Result : PASS

=====
Test Summary
=====
Passed : 4
Failed : 0
Total : 4
Overall Result: ALL TESTS PASSED
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./test_session22.sh > session22_test_report.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cat session22_test_report.txt
Session 22 - Automated Test Report
=====

Test: Hello command
Expected: PASS
Actual : PASS: Hello command recognized
Result : PASS

Test: Test command
Expected: PASS
Actual : PASS: Test command recognized
Result : PASS

Test: Invalid command
Expected: ERROR
Actual : ERROR: Invalid test command
Result : PASS

Test: Unknown input
Expected: ERROR
Actual : ERROR: Unknown input
Result : PASS

=====
Test Summary
=====
Passed : 4
Failed : 0
Total : 4
Overall Result: ALL TESTS PASSED
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```